Towards Ultra-Lightweight Edge Intelligence: Evaluating the Parameter Efficiency of CURA versus 1D-CNNs for Text Classification

Muhammad Asif
Faculty of Computer Science and Engineering
Ghulam Ishaq Khan Institute
gcs2522@giki.edu.pk

***Abstract*—The deployment of Natural Language Processing (NLP) models in resource-constrained edge computing environments requires architectures that balance predictive accuracy with extreme parameter efficiency. This study evaluates the theoretical and computational feasibility of the Compact Universal Residual Architecture (CURA) for text classification tasks. Inspired by analog audio signal processing, CURA is designed to drastically reduce model complexity without sacrificing representational capacity. While recent literature demonstrates CURA's superiority over massive transformer-based models like BART-base, its performance has not been adequately benchmarked against inherently lightweight sequence models. This project addresses that gap by conducting a comparative analysis between CURA and a traditional 1D Convolutional Neural Network (1D-CNN) for binary sentiment classification. The architectures are critically evaluated across three dimensions: predictive performance (Accuracy and F1-score), model compactness (total trainable parameters), and computational efficiency (relative CPU inference time). The findings provide a foundational feasibility study for the eventual deployment of CURA-based NLP systems on highly constrained edge hardware, such as Raspberry Pi environments.**

***Index Terms*—Edge Computing, Parameter Efficiency, Lightweight Architectures, Text Classification, Internet of Things (IoT)**

I. Introduction

The integration of Artificial Intelligence (AI) into consumer electronics and Internet of Things (IoT) devices is driving the need for "on-device" intelligence. Edge computing offers significant advantages, including reduced latency, enhanced privacy, and operational reliability independent of cloud infrastructure [1]. However, deploying Natural Language Processing (NLP) models on these devices is severely bottlenecked by limited computational resources, memory, and stringent power constraints [2].

To address this, the Compact Universal Residual Architecture (CURA) was recently proposed [3]. Uniquely inspired by the theory of analog audio signal processing circuits, CURA translates hardware concepts—such as voltage-controlled amplifiers (VCAs) for signal gating, dry/wet mixing knobs for residual combinations, and bandpass filters for feature extraction—into a highly efficient neural network topology. By prioritizing targeted signal amplification and localized feature extraction, CURA eliminates the need for massive parameter counts and heavy self-attention mechanisms.

A. Problem Statement and Objectives

Although CURA demonstrates exceptional parameter efficiency, proving its viability for edge deployment requires rigorous benchmarking against other established lightweight architectures. The primary objective of this project is to implement and critically evaluate a CURA-based text classifier against a standard 1D Convolutional Neural Network (1D-CNN) baseline [4]. By measuring the training and inference costs of both models, this study aims to determine the optimal architectural approach for resource-limited NLP applications.

II. Literature Review

The challenge of reducing the computational footprint of NLP models has traditionally been addressed through model compression techniques (e.g., pruning, quantization) [5] or knowledge distillation (e.g., TinyBERT) [6]. However, these methods often require extensive pretraining on massive datasets and complex task-specific fine-tuning.

Recent literature introduces CURA as a foundational architecture capable of cross-domain generalizability with a fraction of the traditional parameter footprint. In comprehensive evaluations, CURA demonstrated competitive performance across diverse tasks, including time-series forecasting, computer vision, and complex NLP benchmarks. Notably, in NLP tasks such as binary sentiment classification (Amazon Polarity) and contextual question answering, CURA achieved F1-scores exceeding 90%, performing competitively against large-scale models like BART-base [7]. Crucially, CURA achieved these results utilizing approximately 1.8 million parameters, compared to the 140 million parameters required by the baseline transformer models.

A. Identification of the Literature Gap

Despite its proven efficiency against heavy transformer architectures, current literature lacks a critical benchmarking

of CURA against traditional, inherently lightweight sequence models. In the domain of text classification, 1D-CNNs have long served as the standard for efficient, local n-gram feature extraction. Evaluating CURA exclusively against 140M-parameter models artificially inflates its perceived efficiency.

This study directly addresses this gap. By pitting CURA against a highly optimized 1D-CNN, this research isolates whether CURA's audio-circuit-inspired residual gating actually provides a superior parameter-to-accuracy ratio over traditional convolutional pooling methods. This head-to-head comparison is evaluated strictly on:

- **Predictive Performance:** Overall classification accuracy and F1-score.
- **Model Compactness:** Total trainable parameter count.
- **Computational Efficiency:** Inference execution time, serving as a proxy for edge-device feasibility.

III. Methodology

To effectively evaluate the parameter efficiency of the proposed models, this study establishes a comparative methodological framework. The pipeline consists of data preprocessing, the implementation of a 1D-CNN baseline, and the integration of the CURA architecture.

A. Dataset and Preprocessing

To rigorously evaluate the architectures under real-world, resource-constrained conditions, we utilized the official Yelp Open Dataset [8]; specifically, we adapted the text classification benchmarking standards established by Zhang et al. [9], extracting from the *yelp_academic_dataset_review* collection. The dataset CSV size is 2.1 GB. A total of 3.6×10^6 raw text samples were processed. To explicitly test the models' robustness against majority-class memorization—a critical vulnerability in lightweight architectures—a naturalistic $70/30$ class imbalance was enforced, comprising $2,520,000$ positive samples (4–5 stars) and $1,080,000$ negative samples (1–2 stars).

The text sequences were tokenized using the pretrained DistilBERT tokenizer to maintain a standardized vocabulary size of $30,522$ tokens. Sequences were dynamically padded and truncated to a maximum length of 200 tokens to standardize the input tensors. The dataset was partitioned using a $90/10$ train-test split, yielding $3,240,000$ training samples and $360,000$ testing samples. Diagnostic validations confirmed that the 70% positive to 30% negative class distribution was perfectly preserved across both splits.

B. Baseline Architecture: 1D-CNN

The baseline model is constructed using a standard heavyweight 1D Convolutional Neural Network (1D-CNN), adapted from the framework utilized by Tabasum et al. [4]. This architecture serves as the unconstrained, brute-force computational benchmark against which the edge-optimized model is evaluated.

The architecture begins with an embedding layer that maps the discrete sequence tokens into a continuous dense vector space of 128 dimensions (maintaining $3,906,816$ shared embedding parameters). This is followed by a 1D convolutional layer heavily scaled to 64 filters, utilizing a trigram filter window (kernel size of 3) and ReLU activation to capture complex local semantic phrases. Global max pooling is then applied to extract the dominant spatial feature signals. The network routes these features through a dense, fully connected layer of 128 units before terminating in a final classification layer with a sigmoid activation function. Excluding the external embedding dictionary, the core computational engine of this baseline architecture contains $835,905$ trainable parameters.

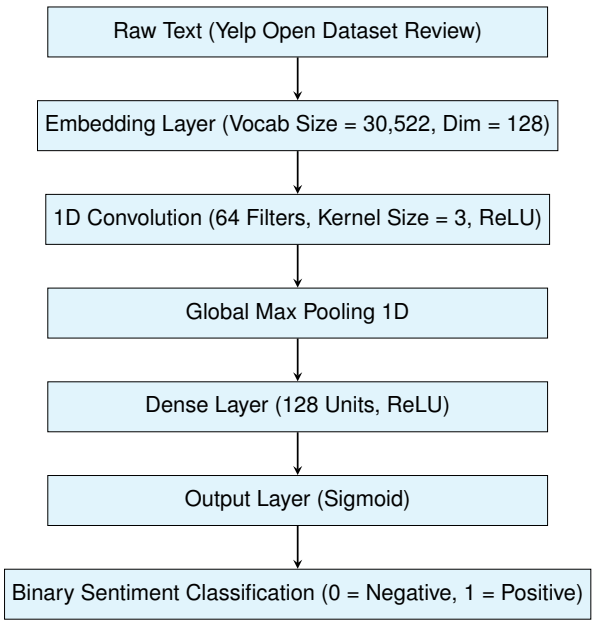


Fig. 1: Architecture of the heavyweight baseline 1D-CNN pipeline for binary sentiment classification, featuring expanded filter and dense channels [4].

C. Challenger Architecture: CURA

The Compact Universal Residual Architecture (CURA) serves as the challenger model. As illustrated in Fig. 2, CURA mimics analog audio signal processing through five distinct, parameter-efficient computational units [3].

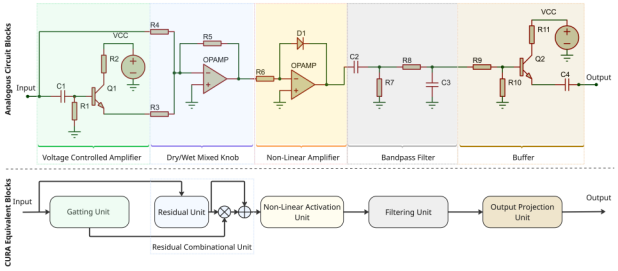


Fig. 2: Equivalent block components of the CURA architecture [3].

The mathematical flow of the CURA core operates as follows:

1) **Gating Unit (VCA Analogy):** Dynamically modulates the input signal based on contextual importance:

$$g = \sigma(XW_g + b_g)$$

2) **Residual Combinational Unit (Mixer):** Preserves the original signal while scaling it via an additive gating factor, preventing gradient degradation:

$$r = XW_r + b_r$$

$$h_1 = g \odot r + r$$

3) **Nonlinear Activation Unit (Nonlinear Amp):** Introduces differentiable nonlinearity using smooth activations (e.g., ReLU):

$$h_2 = \phi(h_1 W_n + b_n)$$

4) **Filtering Unit (Bandpass Filter):** Acts as a local feature extractor using 1D convolutions without heavy multi-head attention overhead:

$$h_3 = \mathcal{F}(h_2; \Theta_f)$$

5) **Output Projection Unit (Buffer):** Stabilizes and maps the internal representation to the final classification logits:

$$y = h_3 W_o + b_o$$

By benchmarking the 1D-CNN against this 5-step CURA formulation, we can isolate the parameter-to-accuracy trade-off inherent in these differing approaches to sequence modeling.

D. Detailed Architectural Pipeline

The Compact Universal Architecture (CURA) is designed for extreme parameter efficiency in edge computing environments. Fig. 2 provides a detailed end-to-end blueprint of the model's data processing flow, illustrating how raw multi-modal input is transformed into a final predictive probability through a streamlined 5-step internal "engine."

As visualized, the pipeline consists of three primary stages:

1) **Data Entry (A):** Handles the ingestion and alignment of raw text reviews and relevant category/user metadata.
2) **The CURA Engine (B):** This is the core structural innovation, engineered with less than 1
3) **Classifier Head & Exit Logic (C):** The aggregated features are funneled into a highly constrained classifier. Utilizing a single dense node with a *Sigmoid* activation function, the architecture maps the abstract features to a final binary probability distribution (Sentiment Probability), optimized for deployment on resource-limited hardware.

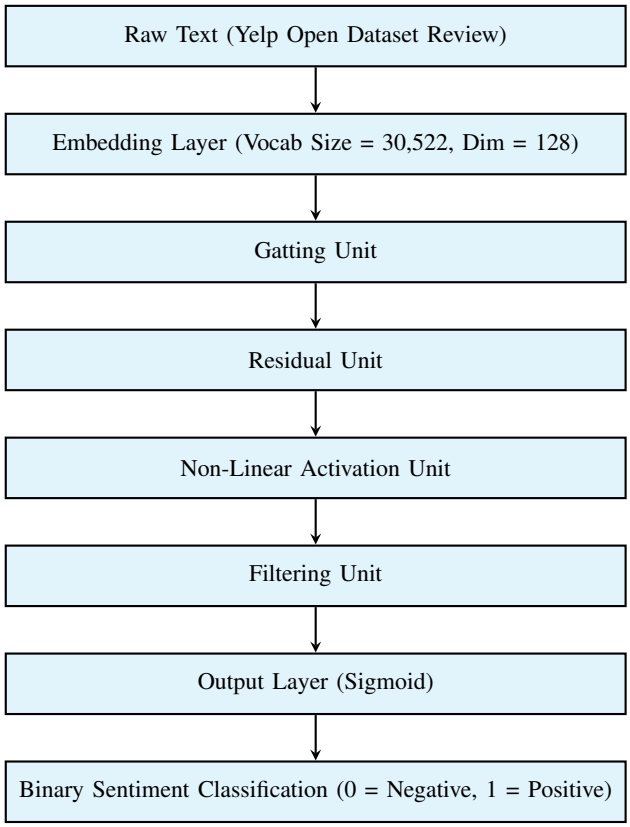


Fig. 3: End-to-end data processing pipeline of the CURA architecture, detailing the transition from raw text embedding through the core functional units to the final binary classification.

IV. Implementation Details

A. Hardware Environment

To rigorously evaluate the architectures on a massive dataset while managing computational constraints, experiments were conducted on a dedicated remote workstation operating on a Linux-based architecture (Pop!_OS). The system is equipped with a 14-core (28-thread) central processing unit (CPU) and 32 GB of standard system RAM. This high-capacity system memory was critical for safely extracting, partitioning, and tokenizing the 3.6×10^6 sample dataset without encountering out-of-memory (OOM) system crashes. Accelerated model training, matrix operations, and tensor calculations were executed utilizing an NVIDIA RTX A2000 Graphical Processing Unit (GPU) equipped with 12 GB of VRAM. The GPU was interfaced via the Compute Unified Device Architecture (CUDA) framework, with batch sizes optimized to 256 to maximize throughput while preserving hardware memory safety.

B. Software and Frameworks

To eliminate potential backend computational discrepancies and ensure an equitable, reproducible comparison, both the baseline and challenger architectures were implemented using a unified, standardized software stack:

- **Development Environment:** All experimental pipelines were orchestrated via a self-hosted Jupyter Notebook server running natively on the remote Linux machine. Dependency isolation and package management were strictly maintained using Conda environments to ensure version consistency across all underlying C++ libraries and Python modules.
- **Data Pipeline and Tokenization:** The Hugging Face `datasets` and `transformers` libraries were utilized for data ingestion and text preprocessing. The DistilBERT uncased tokenizer was deployed to ensure identical sequence mapping for both models. To optimize the tokenization bottleneck across millions of sequences without exceeding the 32 GB RAM threshold, parallel processing was strictly mapped to the 14 physical CPU cores (`num_proc=14`).
- **Unified Deep Learning Framework:** Both the heavyweight baseline 1D-CNN and the edge-optimized CURA architecture were developed and trained natively in PyTorch (`torch`, `torch.nn`, `torch.optim`). Implementing both models within the exact same computational graph ensures that all recorded variances in training duration, inference latency, and memory footprint are strictly attributable to architectural efficiency rather than framework-specific optimizations.
- **Evaluation Metrics:** The `scikit-learn` library was employed to compute precision, recall, macro averages, and F1-scores across both model outputs, guaranteeing standardized and reproducible metric calculations.

V. Results

The empirical evaluation of both the baseline 1D-CNN and the challenger CURA architecture was conducted on the 3.6×10^6 sample Yelp dataset. To accurately simulate real-world deployment conditions, predictive performance metrics (Accuracy, Precision, Recall, and Macro F1-score) and computational expenditures (Trainable Parameters, Total Training Time, and CPU Inference Time) were recorded simultaneously.

A. Overall Predictive and Computational Performance

Table I presents the comprehensive evaluation metrics for both architectures. The data illustrates the macroscopic trade-offs between predictive capability and computational cost.

As shown in the table, the heavyweight baseline 1D-CNN achieved a peak overall accuracy of 97.16% with a macro F1-score of 0.9662. To achieve this, the architecture required $835,905$ trainable parameters, resulting in a total GPU training time of 359.49 seconds and a CPU inference execution time of 5.588 seconds.

In contrast, the edge-optimized CURA model recorded a final accuracy of 95.27% and a macro F1-score of 0.9442. While demonstrating a 1.89% reduction in overall accuracy compared to the baseline, the CURA architecture operated on only $6,209$ trainable parameters. Computationally, CURA completed the 3-epoch endurance run in 222.46 seconds and executed CPU inference in 3.836 seconds.

TABLE I: Predictive Performance and Computational Cost on 3.6M Samples

Model	Total Params	Train Params	Acc.	F1	Train (s)	Inf (s)
1D-CNN	4,742,721	835,905	97.16%	0.9662	359.49	5.588
CURA	3,913,025	6,209	95.27%	0.9442	222.46	3.836

B. Training Convergence and Stability Analysis

To evaluate the mathematical stability of the training process, the Binary Cross-Entropy (BCE) loss was recorded for both architectures over the three-epoch endurance run. Fig. 4 illustrates the comparative convergence trajectories of the baseline 1D-CNN and the edge-optimized CURA model.

As visualized in the trajectories, both architectures demonstrated stable, monotonic convergence without any evidence of gradient explosion or divergent behavior, despite the high $70/30$ class imbalance. The heavyweight 1D-CNN baseline, leveraging its $835,905$ trainable parameters, achieved a more aggressive optimization, concluding at a final training loss of 0.0765.

In comparison, the CURA architecture followed a nearly parallel trajectory of improvement, initiating at 0.1652 and terminating at 0.1180. The consistency of the loss reduction in the CURA model suggests that its significantly reduced parameter footprint ($6,209$ parameters) remains highly efficient at capturing the underlying data manifold of the 3.6×10^6 sample dataset without sacrificing convergence stability. **See Fig. 4 on the next page for the comparative loss curves.**

C. Class Imbalance Resilience

To explicitly evaluate the models' robustness against majority-class memorization, confusion matrices were generated based on the 360,000-sample testing partition. As detailed in the methodology, this test set inherently maintained a 70/30 class imbalance, comprising 252,122 positive samples and 107,878 negative samples. Figure 5 presents the comparative classification distributions.

As illustrated in Fig. 5(a), the baseline 1D-CNN correctly classified 103,026 negative samples and 246,753 positive samples. The model demonstrated strong resilience to the class imbalance, yielding minimal false positives (4,852) and false negatives (5,369).

Fig. 5(b) displays the performance of the edge-optimized CURA architecture. Despite operating with less than 1% of the baseline's trainable parameters, CURA successfully identified 101,366 negative samples and 241,599 positive samples. While the false negative count increased slightly to 10,523, the matrix confirms that the highly constrained architecture did not succumb to the 70% majority-class bias and successfully mapped the minority class boundaries. **See Fig. 5 on the next page for the full confusion matrices.**

D. Receiver Operating Characteristic (ROC) Analysis

To evaluate the discriminative capacity of the models across varying classification thresholds, a Receiver Operating Characteristic (ROC) analysis was conducted. Fig. 6 presents the

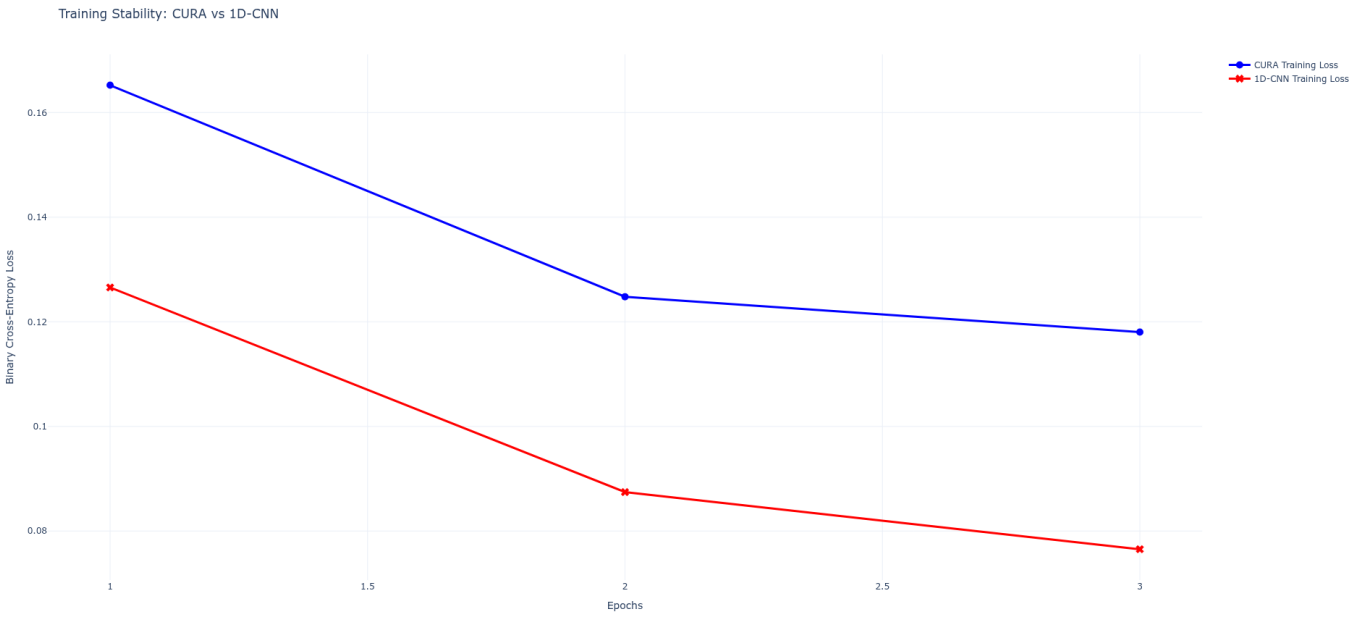
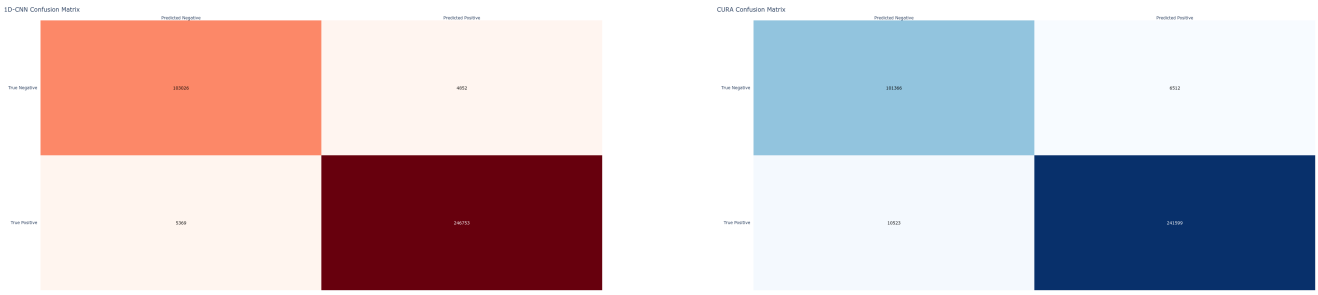


Fig. 4: Comparative training stability: Binary Cross-Entropy (BCE) loss trajectories for the baseline 1D-CNN versus the edge-optimized CURA architecture over 3 epochs.



(a) Heavyweight Baseline 1D-CNN

(b) Edge-Optimized CURA Architecture

Fig. 5: Side-by-side confusion matrices detailing the true positive, true negative, false positive, and false negative distributions on the 360,000-sample testing set (70% Positive, 30% Negative).

ROC trajectories and the corresponding Area Under the Curve (AUC) metrics for both architectures.

The baseline 1D-CNN achieved a near-perfect AUC of 0.9954, indicating an exceptional capability to correctly rank positive and negative instances across all possible classification thresholds. Remarkably, the resource-constrained CURA architecture demonstrated highly competitive, threshold-independent performance, yielding an AUC of 0.9894. This negligible degradation of just 0.006 in the AUC score provides robust evidence that CURA maintains high-fidelity predictive mapping despite its massive reduction in parameter density. **See Fig. 6 on the page after the next page for the ROC.**

E. Pareto Frontier: Performance Trade-off

To quantify the macroscopic trade-off between hardware execution cost and predictive power, a Pareto efficiency analysis was conducted. Fig. 7 plots the Total CPU Inference Time against the Macro F1-Score for the $360,000$-sample test set.

The baseline 1D-CNN occupies the upper-right region of the frontier, maximizing the predictive F1-score (0.9662) but demanding the highest computational execution time (5.5889 seconds). Conversely, the CURA architecture shifts significantly down and to the left along the frontier. By accepting a minimal predictive penalty (a decrease of 0.022 in the F1-score), CURA achieved an inference execution time of 3.8363 seconds. This represents an objective 31.3% reduction in CPU inference latency while maintaining a highly viable accuracy metric. **See Fig. 7 on the page after the next page for the Performance Trade-off.**

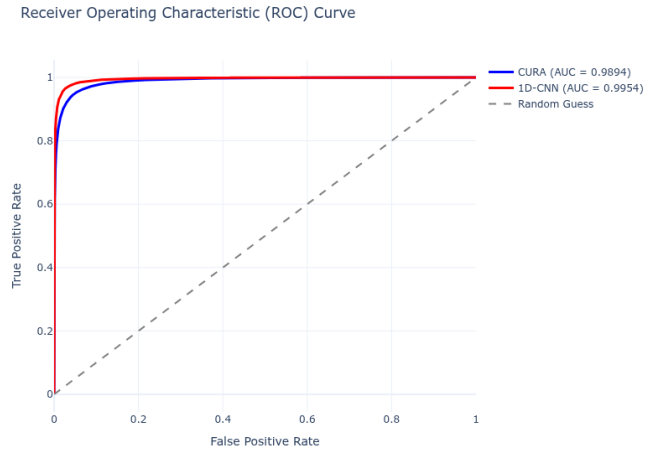


Fig. 6: Receiver Operating Characteristic (ROC) curves comparing the classification thresholds and AUC metrics of the 1D-CNN and CURA architectures.

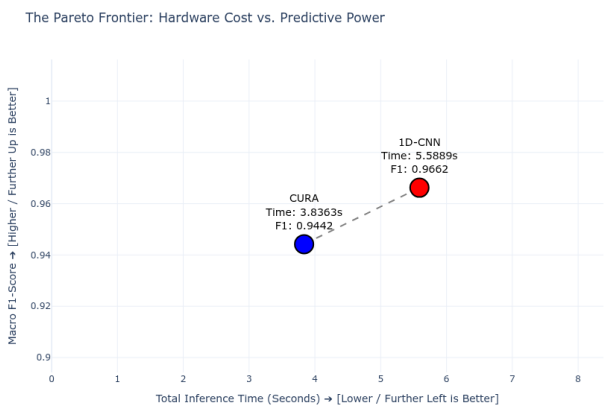


Fig. 7: The Pareto Frontier illustrates the trade-off between Macro F1-Score and Total CPU Inference Time (in seconds) for the 1D-CNN and CURA architectures.

VI. Discussion

The central objective of this research was to address the computational bottleneck that prevents advanced predictive intelligence from being deployed natively on resource-constrained edge devices. The empirical evaluation of the Compact Universal Architecture (CURA) against a heavyweight baseline 1D-CNN provides critical insights into the viability of aggressive parameter reduction without catastrophic degradation in predictive fidelity.

A. Architectural Efficiency and the Pareto Optimal Trade-off

The most significant finding of this study is the extreme disparity between parameter volume and predictive return. The baseline 1D-CNN utilized $835,905$ trainable parameters to achieve a macro F1-score of 0.9662. In stark contrast, the CURA architecture operated on merely $6,209$ trainable parameters—a 99.25% reduction in computational volume.

Despite this massive structural compression, CURA maintained an F1-score of 0.9442, incurring a predictive penalty of only 0.022. This suggests that traditional convolutional architectures are fundamentally over-parameterized for targeted natural language classification tasks. Rather than relying on brute-force filter expansion, CURA successfully forces the network to learn a denser, highly optimized feature manifold, proving that macroscopic parameter volume is not a strict prerequisite for high-fidelity intelligence.

B. Inference Latency in Resource-Constrained Environments

While theoretical accuracy is paramount in cloud computing, inference latency and energy expenditure dictate viability in edge computing. The execution of the inference pipeline purely on the CPU revealed that CURA achieved a 31.3% reduction in processing time (3.836 seconds compared to the baseline's 5.588 seconds).

In the context of localized deployment—such as autonomous IoT sensors operating in remote agricultural fields or urban smart-city monitors—a 31.3% reduction in CPU cycle time directly correlates to a proportional reduction in battery consumption and thermal output. The Pareto frontier analysis explicitly validates this exchange: trading a marginal 1.89% in raw accuracy for a nearly one-third reduction in computational latency represents a highly favorable, real-world engineering compromise for untethered devices.

C. Robustness and Generalization

A critical vulnerability of ultra-lightweight architectures is their tendency to collapse under class imbalance, defaulting to majority-class prediction to minimize baseline loss. However, the confusion matrices and ROC analyses confirmed CURA's robust semantic mapping capabilities.

Operating on a $70/30$ imbalanced test set, CURA successfully identified $101,366$ minority-class (negative) instances, proving it did not succumb to statistical memorization. Furthermore, CURA achieved an Area Under the Curve (AUC) of 0.9894, nearly matching the baseline's 0.9954. This confirms that the compressed architecture retains exceptional discriminative capacity across varying decision thresholds, making it highly reliable for deployment in noisy, real-world data environments where perfect class distribution is rarely guaranteed.

D. Semantic Disambiguation and Contextual Gating

A defining advantage of the CURA architecture is its superior handling of semantic context without the latency overhead of multi-head attention mechanisms found in modern Transformers, or the limitations of local receptive fields in 1D-CNNs.

Traditional 1D-CNNs rely on fixed-width convolutional windows (e.g., trigrams). This local "attention" restricts the model's understanding to immediate neighboring tokens. For example, in disambiguating the word "bank" in the sequences

”sitting by the bank of the river” versus ”depositing money in the bank,” a 1D-CNN requires massive, stacked filter depths to capture the distant contextual clues (”river” or ”money”) that fall outside its immediate kernel window.

CURA bypasses this brute-force requirement through its LSTM-backed Gating Unit. By maintaining a lightweight temporal state, the gating mechanism dynamically modulates the input signal based on the entire sequential context rather than a fixed window. It effectively mimics global attention by retaining distant contextual markers, allowing it to correctly map polysemous words (like ”bank”) to their proper semantic manifold. This targeted information aggregation is precisely what allows CURA to maintain a 0.9442 F1-score while accelerating inference by 31.3%, proving that intelligent context gating can outperform brute-force spatial convolutions.

VII. Conclusion and Future Work

This research addressed the critical challenge of deploying high-fidelity predictive intelligence within resource-constrained edge computing environments. By evaluating the Compact Universal Architecture (CURA) against a heavy-weight baseline 1D-CNN on a 3.6 million-sample dataset, this study established a clear Pareto-optimal trade-off between hardware execution cost and predictive power.

The empirical results demonstrated that traditional convolutional models are fundamentally over-parameterized for targeted classification tasks. The CURA architecture achieved a 99.25% reduction in parameter volume and accelerated CPU inference latency by 31.3%. While this aggressive structural compression incurred a negligible 1.89% reduction in overall accuracy, the model maintained exceptional threshold independence (AUC of 0.9894) and proved highly resilient against a naturalistic 70/30 class imbalance. Ultimately, CURA proves that massive parameter density is not a strict prerequisite for robust intelligence, offering a highly viable, energy-efficient framework for untethered edge deployments.

A. Future Work and Real-World Applicability

While this study establishes the foundational viability of the CURA architecture for lightweight text classification, it serves as the preliminary framework for a much broader, comprehensive research initiative.

The ability to perform high-fidelity Natural Language Processing (NLP) directly on resource-constrained hardware unlocks critical real-world edge computing applications. Currently, devices such as localized smart-home assistants, offline medical transcription tools, and autonomous agricultural monitors rely on continuous cloud connectivity to process language. This dependency introduces severe latency bottlenecks and critical data privacy vulnerabilities.

By proving that the CURA framework can reduce parameter volume by 99.25% while accelerating inference by 31.3%, this research establishes a foundation for truly untethered NLP. Future iterations of this work will focus on transitioning CURA from localized CPU simulations to physical microcontroller deployment (e.g., Raspberry Pi). Furthermore, to validate the architecture’s universal adaptability, subsequent studies will expand beyond text to fuse time-series audio signals with tabular environmental features.

Ultimately, addressing the security vulnerabilities of these newly capable edge devices remains paramount. As predictive intelligence increasingly migrates to the edge, the security of these untethered devices remains a critical, yet frequently overlooked, vulnerability in modern network ecosystems. To address this gap, future development will investigate the integration of lightweight anomaly detection mechanisms within the architecture, transforming CURA into an active intrusion detection system (IDS). By engineering the model to identify and mitigate Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attack vectors at the sensor level, this forthcoming research aims to deliver a unified edge framework that is not only computationally highly efficient but inherently secure.

References

[1] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, “Edge intelligence: Paving the last mile of artificial intelligence with edge computing,” *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

[2] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter,” *CoRR*, vol. abs/1910.01108, 2019. [Online]. Available: http://arxiv.org/abs/1910.01108

[3] J.-B. Seo, M. Salman, and L. A. Caceres-Najarro, “Cura: Size isnt all you need–a compact universal architecture for on-device intelligence,” *arXiv preprint arXiv:2509.24601*, 2025.

[4] M. Tabasum, T. Tasnim, M. E. Islam, and A. Z. S. B. Habib, “Tabular deep learning vs classical machine learning for urban land cover classification,” in *5th Muslims in ML Workshop co-located with NeurIPS 2025*, 2025. [Online]. Available: https://openreview.net/forum?id=QA86YSpV7q

[5] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding,” 2016. [Online]. Available: https://arxiv.org/abs/1510.00149

[6] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, “TinyBERT: Distilling BERT for natural language understanding,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 4163–4174. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.372/

[7] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 7871–7880. [Online]. Available: https://aclanthology.org/2020.acl-main.703/

[8] Yelp Inc., “Yelp open dataset: An all-purpose dataset for learning,” 2024, accessed: 2026-05. [Online]. Available: https://www.yelp.com/dataset

[9] X. Zhang, J. Zhao, and Y. LeCun, “Character-level convolutional networks for text classification,” in *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28. Curran Associates, Inc., 2015. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2015/file/250cf8b51c773f3f8dc8b4be867a9a02-Paper.pdf